UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



LAB 03 - GitHub Spec Kit

CURSO:

TOPICOS AVANZADOS EN INGENIERIA DE SOFTWARE

DOCENTE:

ROBERT EDISON ARISACA MAMANI

PRESENTADO POR:

JEAN CARLO CHARA CONDORI

VLADIMIR JAWARD HANCCO SONCCO

GONZALO RAIL LAYME MAMANI

JEANPIER MICHAELSON VALERA YANA

AREQUIPA – PERÚ

2026

Contexto del proyecto aplicado

El sur del Perú se ubica en una región de intensa actividad sísmica interplaca. Durante eventos telúricos de gran magnitud, la infraestructura de telecomunicaciones comerciales y el suministro eléctrico colapsan en los primeros minutos debido a la congestión de canales o la caída de antenas base. En este escenario crítico, la población civil queda desprovista de conectividad a la nube y requiere herramientas digitales inmediatas, confiables y completamente autónomas para coordinar su seguridad familiar.

Con el objetivo de resolver esta problemática, se desarrolló **SismoListo**, una aplicación web progresiva orientada bajo una política estricta de **100% offline-first**. La plataforma integra dos módulos fundamentales de preparación ante desastres:

1. **Mochila de Emergencia:** Checklist interactivo estructurado según las 6 categorías oficiales establecidas por el Instituto Nacional de Defensa Civil (INDECI), con cálculo reactivo inmediato del porcentaje de preparación en el intervalo cerrado $[0, 100]\%$.
2. **Plan Familiar de Emergencia:** Registro y gestión de hasta 20 integrantes del núcleo del hogar con asignación de roles operativos, definición de puntos de reunión seguros (internos y externos) y accesos directos al servicio oficial de mensajería de voz 119.

En términos metodológicos, el proyecto se construyó aplicando el paradigma de **Spec-Driven Development (SDD)** formalizado por Piskala [1], utilizando la herramienta oficial **GitHub Spec Kit** [2] integrada con el agente inteligente **Antigravity CLI (agy)**. El ciclo completo de especificación, arquitectura, desglose de tareas, implementación modular y verificación matemática se encuentra documentado y publicado en el repositorio oficial:

https://github.com/vhanccos/tais_lab03_grupal

Configuración del Entorno

En el marco de SDD, el desarrollo no inicia con la redacción apresurada de código ni con prompts desestructurados, sino con la configuración formal de la infraestructura de especificación ejecutable. En el directorio raíz del proyecto (`~/tmp/taislab/lab03`), se inicializó el repositorio Git y se ejecutó la CLI de GitHub Spec Kit vinculada al agente Antigravity:

Terminal — Inicialización de GitHub Spec Kit con integración AGY — Shell

```
1  git init
2  uvx --from git+https://github.com/github/spec-kit.git specify init --here --force --non-interactive --integration agy
```



Figura 1. Inicialización de GitHub Spec Kit en consola con el adaptador agy y despliegue del andamiaje SDD.

Esta operación aprovisionó dos directorios fundamentales: `.specify/`, que almacena las plantillas estructuradas de requerimientos, planes técnicos, tareas y directivas de control; y `.agents/`, que registra las habilidades operativas agénticas (`/speckit-specify`, `/speckit-plan`, `/speckit-tasks` e `/speckit-implement`).

Fase 1: Especificación

Generación de la Especificación

Se ejecutó la habilidad `/speckit-specify` proporcionando la descripción del dominio de sismos. El agente generó la rama de trabajo `001-sismolisto-offline` y produjo el documento formal `specs/001-sismolisto-offline/spec.md`, estructurado en historias de usuario con criterios de aceptación en formato Given/When/Then, entidades clave y criterios de éxito medibles.



Figura 2a. Prompt de entrada para /speckit-specify.

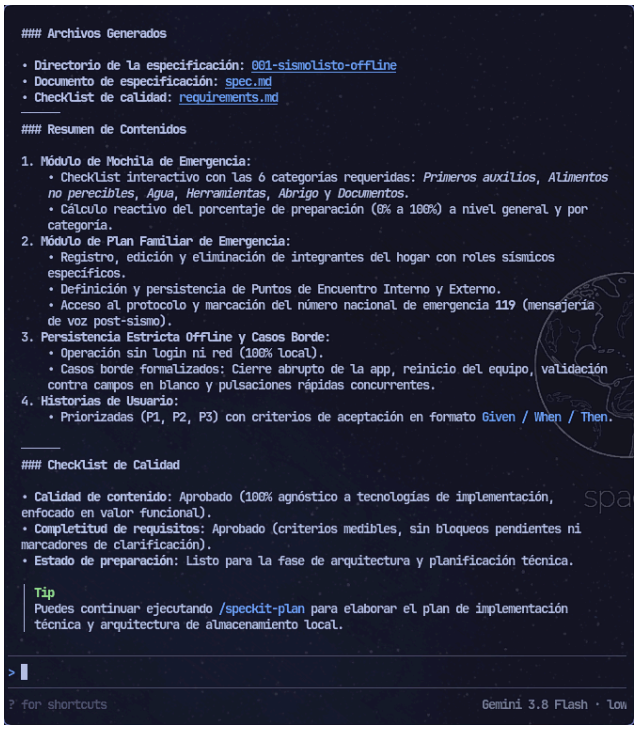


Figura 2b. Finalización y síntesis agéntica.

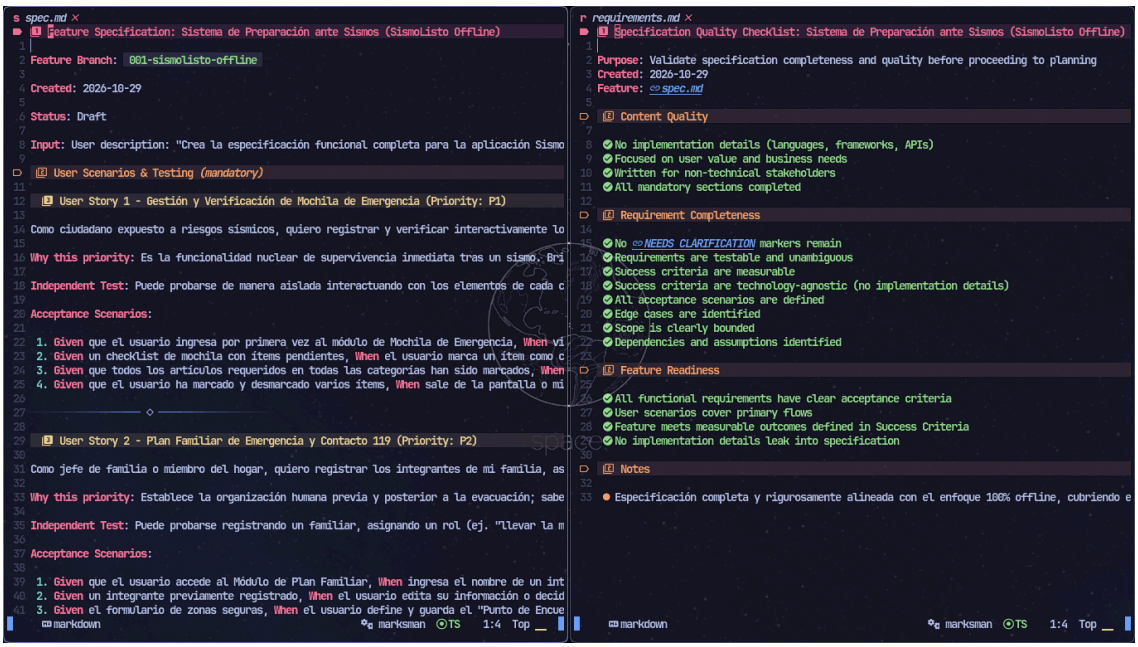


Figura 3. Artefactos funcionales generados: spec.md y matriz de requerimientos.

Revisión de Dominio y Correcciones

En estricto apego al principio de Piskala [1] donde la especificación manda sobre el código, se realizó una auditoría manual exhaustiva de `spec.md`, detectando dos inconsistencias críticas de dominio que el modelo pasó por alto:

1. **Sintaxis Real del Protocolo 119:** La versión inicial proponía un botón de marcación directa a `tel:119`. En la red telefónica peruana, el 119 no es una línea de voz convencional; es un sistema de mensajería con sintaxis obligatoria: `1191{celular}` para grabar y `1192{celular}` para escuchar. Se corrigió la especificación exigiendo un campo de 9 dígitos para el número familiar clave y dos botones diferenciados de acción rápida.
2. **Invariante Matemático ante Lista Vacía:** Si el usuario borraba o desmarcaba todos los artículos de la mochila, la fórmula `(completados / total)` generaba una división por cero indeterminada (`0/0 = NaN`). Se incorporó el invariante formal: si el total de artículos es 0, el progreso retornado debe ser estrictamente 0%, acotado en el intervalo discreto $[0, 100]$.

Ambas correcciones fueron validadas y consolidadas en Git mediante el commit: `4bc940b docs(specs): corregir protocolo de marcación 119 e invariantes de cálculo de progreso`.

Fase 2: Planificación Técnica

Arquitectura y Contratos

Se invocaron `/speckit-plan` y `/speckit-tasks`. El agente generó `plan.md`, `research.md`, `data-model.md` y los contratos formales en `contracts/`, definiendo una arquitectura desacoplada para frontend basada en TypeScript, React 19, Vite, Tailwind CSS y persistencia local pura sobre `localStorage` con fallback en memoria.

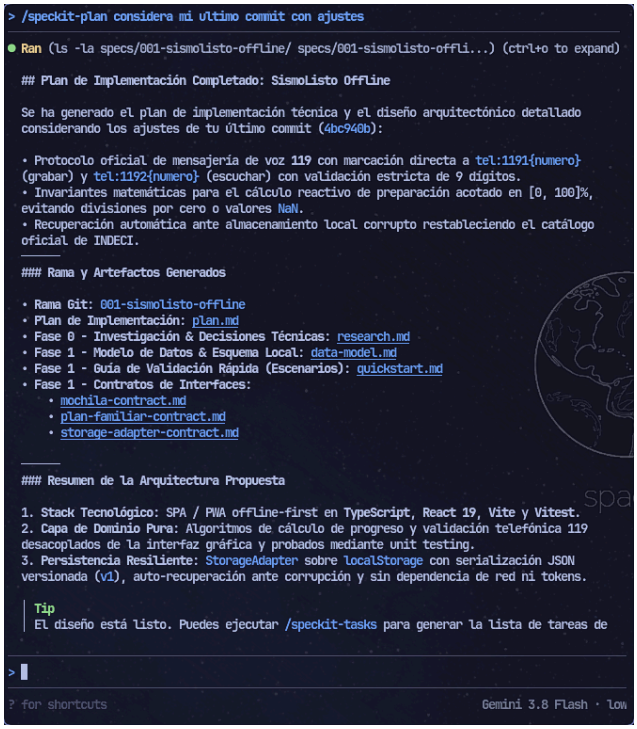


Figura 4a. Ejecución de /speckit-plan en AGY.

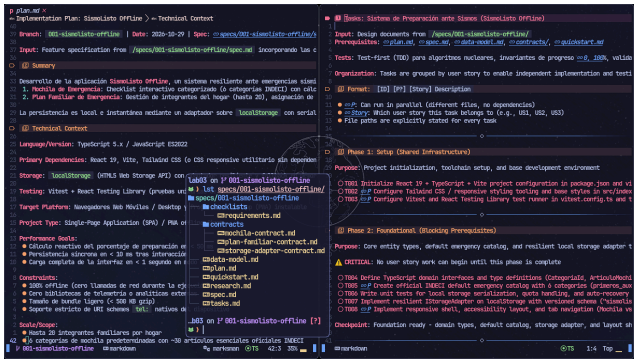


Figura 4b. Artefactos plan.md, tasks.md y estructura.

Incorporación de Property-Based Testing

Durante la auditoría del plan técnico se observó que la estrategia de pruebas formulada por el agente se limitaba a pruebas unitarias clásicas basadas en ejemplos discretos (`tests/unit/`).

Ajuste Incorporado: Se exigió la inclusión formal de **Property-Based Testing (PBT)** mediante la biblioteca **Fast-Check**. A diferencia de un test unitario que prueba combinaciones arbitrarias como «3 de 10 completados = 30%», PBT evalúa invariantes universales sobre espacios aleatorios masivos:

- **Propiedad 1 (Invariante de Progreso):** Para cualquier lista aleatoria de N artículos con cualquier distribución booleana de completado, el progreso siempre debe ubicarse en $[0, 100]$ y jamás ser `NaN`.
- **Propiedad 2 (Round-Trip de Persistencia):** Todo estado serializado en almacenamiento local debe recuperarse de forma idéntica sin pérdida de atributos.

Las tareas `T003`, `T006b`, `T009b` y `T017` fueron agregadas a `tasks.md`, quedando confirmadas en Git mediante el commit: `7c4d589 docs(tasks): incorporar tareas de Property-Based Testing (Fast-Check) y casos de borde de marcación 119.`

Fase 3: Implementación

La implementación se ejecutó mediante `/speckit-implement`, construyendo el sistema modularmente desde el núcleo de dominio hacia la interfaz:

1. **Núcleo Puro (`src/core/`):** `defaultCatalog.ts` contiene los 30 insumos oficiales de INDECI agrupados en 6 categorías. `progressCalculator.ts` implementa el algoritmo invariante del progreso general y por categoría. `phoneValidator.ts` valida celulares peruanos de 9 dígitos y construye las URIs `tel:1191{numero}` y `tel:1192{numero}`.
2. **Adaptador de Persistencia (`src/storage/storageAdapter.ts`):** Gestiona la clave versionada `sismolisto_app_state_v1`. Si `localStorage` arroja una excepción de cuota excedida o JSON corrupto, el adaptador auto-recupera la aplicación restaurando el catálogo oficial y manteniendo una copia en memoria sin interrumpir la experiencia.
3. **Capa de Presentación (`src/components/`):** `ChecklistMochila.tsx` implementa acordiones accesibles por categoría y una barra de progreso reactiva. `PlanFamiliarView.tsx` integra la gestión de familiares con límite de 20 personas, zonas seguras y los botones de marcación 119.



Figura 5. Finalización de la implementación en AGY y estructura de archivos fuente.

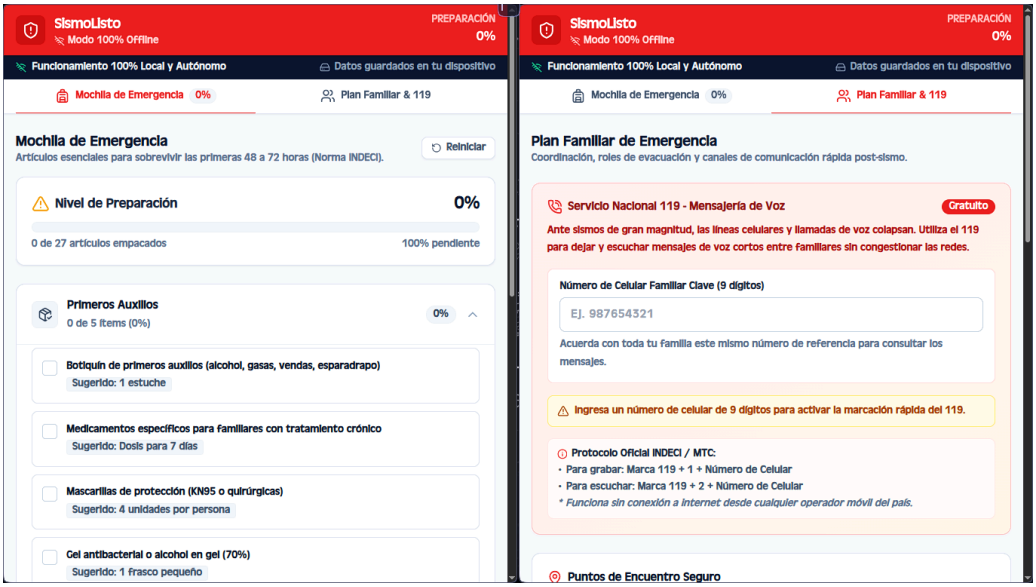


Figura 6. Interfaz de SismoListo: Checklist interactivo de la mochila (izquierda) y Plan Familiar con acceso 119 (derecha).

Revisión de Persistencia Reactiva

Durante la auditoría del código generado en la Fase 3, se detectó una discrepancia de diseño en el componente `ZonasSeguras.tsx`. El agente implementó un estado local desacoplado (`useState`) que requería hacer clic manual en un botón para almacenar los datos. Este patrón convencional de formulario web contravenía directamente los requerimientos funcionales y escenarios de la especificación:

- **FR-006 & User Story 2 (Criterio 3):** El sistema debe garantizar el registro y disponibilidad continua de las zonas de encuentro seguras (interna y externa).
- **FR-009 & User Story 3 (Criterio 2):** Se exige persistencia instantánea y automática de cualquier cambio en el almacenamiento local sin requerir confirmaciones manuales. Bajo la implementación inicial, si el usuario definía su punto de reunión y cerraba la aplicación o el dispositivo se apagaba por el sismo, los datos se destruían silenciosamente.

Para garantizar la integridad operativa ante emergencias, se refactorizó el componente eliminando el botón superfluo y vinculando manejadores reactivos inmediatos (`onChange`) conectados directamente al contexto de persistencia. Asimismo, se instaló formalmente la dependencia `fast-check` en `package.json` para habilitar las suites de pruebas de propiedades planificadas en la Fase 2. El parche se registró con el commit: `c299d87 fix(review-3): implementar auto-guardado reactivo de zonas seguras y suites de Property-Based Testing (Fast-Check).`

Fase 4: Validación

La verificación combinó tres niveles de pruebas ejecutadas de forma automatizada mediante Vitest:

1. **Pruebas Unitarias de Dominio y Componentes:** Se verificó la lógica de cálculo (`progressCalculator.test.ts`), la validación telefónica (`phoneValidator.test.ts`), los mecanismos de auto-recuperación ante fallos de disco (`storageAdapter.test.ts`) y la interacción visual de componentes (`ChecklistMochila.test.tsx` y `PlanFamiliar.test.tsx`).
2. **Prueba de Integración Offline (`offlineLifecycle.test.tsx`):** Simuló el ciclo de vida completo de la aplicación, ejecutando montajes, modificaciones de datos, desmontajes abruptos del árbol DOM y recargas completas sin conectividad, constatando cero pérdida de información.
3. **Pruebas Basadas en Propiedades (`tests/property/`):** Con Fast-Check (100 ejecuciones por propiedad), `progressInvariants.pbt.test.ts` evaluó listas con tamaños arbitrarios de 0 a 500 ítems, confirmando matemáticamente que el progreso siempre se acota en $[0, 100]$ y jamás produce `NaN`. Paralelamente, `storageRoundTrip.pbt.test.ts` verificó la integridad serializable del estado global.



Figura 7. Resultados de la suite Vitest: 8 archivos y 21 pruebas aprobadas al 100% (Unitarias + PBT).

Los 8 archivos de prueba y las 21 suites pasaron de forma unánime y limpia, sin alertas espurias de consola y garantizando que el build de producción (`npm run build`) compile sin advertencias de tipos.

Discusión

Análisis de Observaciones por Fase

Durante la auditoría humana realizada en cada compuerta del flujo SDD, se identificaron discrepancias sistemáticas entre lo generado por el agente y los requisitos reales del sistema:

1. **Fase 1 (Especificación):** El agente propuso una llamada directa genérica para el 119 y omitió la indeterminación aritmética ($\frac{0}{0} = \mathrm{NaN}$) ante una lista vacía. Esto evidencia que los modelos tienden a asumir flujos estándar genéricos, desatendiendo normativas locales peruanas y casos de borde matemáticos si no se auditan tempranamente.
2. **Fase 2 (Planificación):** El plan inicial contemplaba únicamente pruebas basadas en ejemplos discretos (**happy paths**). Fue indispensable incorporar Property-Based Testing (Fast-Check) en las tareas para garantizar la exploración de combinaciones adversariales y la verificación de invariantes globales.
3. **Fase 3 (Implementación):** Se introdujo persistencia manual mediante botón en `ZonasSeguras.tsx`. En contextos de contingencia y aplicaciones offline-first, la persistencia reactiva inmediata (`onChange`) es crítica para evitar la pérdida de datos ante cierres abruptos de la aplicación.
4. **Fase 4 (Validación):** La auto-recuperación ante fallos de disco operaba adecuadamente, pero arrojaba trazas de error sin silenciar en consola. La validación permitió constatar la resiliencia en memoria y asegurar una ejecución limpia sin alertas residuales.

Control de Versiones y Pipeline con Git

En el estado del arte actual, herramientas como GitHub Spec Kit operan como generadores sobre el sistema de archivos local, dejando el control de versiones a la disciplina manual del desarrollador. En flujos agénticos multi-fase donde intervienen modelos probabilísticos, la falta de puntos de control atómicos incrementa el riesgo de arrastre de errores (**cascading hallucinations**): una omisión en la especificación se traduce en contratos deficientes y finalmente en código defectuoso, escalando el costo de corrección.

Propuesta: Pipeline con Git

1. **Guardia Pre-Ejecución (`pre-spec guard`):** Antes de ejecutar cualquier `/speckit-*`, verificar que el árbol de trabajo esté limpio (`git status --porcelain`). Si existen cambios sin confirmar, abortar para evitar contaminación cruzada. Validar con `git log -1` que el commit previo corresponda a la fase antecedente aprobada.
2. **Commit Atómico Post-Fase (`post-spec commit`):** Tras generar el artefacto (`spec.md`, `plan.md` o `tasks.md`), el agente debe registrar automáticamente un commit convencional acotado a dichos archivos.

Verificación Formal con Property-Based Testing

Cuando un agente de IA genera en la misma sesión el código y las pruebas unitarias basadas en ejemplos, se produce un sesgo de confirmación: el agente evalúa su propio software utilizando los mismos supuestos, simplificaciones y casos favorables con los que codificó.

La integración de Property-Based Testing (Fast-Check) elimina esta limitación. En lugar de verificar casos puntuales arbitrarios, somete las funciones a cientos de miles de entradas generadas pseudoaleatoriamente en busca de contraejemplos mínimos (**shrinking**). En sistemas críticos como SismoListo, esto certificó formalmente la cota matemática $[0, 100]\%$ ante cualquier combinación de ítems y la reversibilidad exacta de la persistencia local.

Referencias

- [1] D. B. Piskala, «Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants,» Seattle, USA, Tech. Rep. arXiv:2602.00180v1, Ene. 2026. [En línea]. Disponible en: https://arxiv.org/abs/2602.00180.
- [2] GitHub, «Spec-Driven Development with AI: Get Started with a New Open Source Toolkit,» GitHub Blog, 2025. [En línea]. Disponible en: https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/.
- [3] Instituto Nacional de Defensa Civil (INDECI), «Plan Familiar de Emergencia: La Seguridad Empieza en Casa», Plataforma Digital Única del Estado Peruano, Lima, Perú, 2023.
- [4] Ministerio de Transportes y Comunicaciones (MTC), «Protocolo de Operación del Servicio de Mensajería de Emergencia 119 ante Desastres», Diario Oficial El Peruano, Lima, Perú.
- [5] C. Baier y J.-P. Katoen, «Principles of Model Checking», MIT Press, Cambridge, MA, 2008.